

Overview

For this assignment, I built a Boggle solver that finds all valid words on a board using a dictionary.

My solver uses DFS with backtracking, explores all eight directions and keeps track of used tiles so letters are not reused in the same word.

To make it faster, I use a prefix set to stop searching paths that can't possibly form words. The solver returns all words with three or more letters sorted alphabetically and ignores case.

When I first wrote the solver, I focused on correctness and clarity, making sure edge cases like empty grids and "Qu" tiles were handled.

After finishing, I did a peer review with my partner to see what could be improved.

Feedback I Got

My partner thought my solver was solid and correctly implemented DFS + backtracking with prefix pruning.

Key takeaways from their feedback:

1. Style and readability: They suggested cleaning up comments, fixing small typos and standardizing indentation.
2. Reducing repeated code: They noticed I was repeating the dictionary and prefix setup in multiple places and suggested putting it in a helper function.
3. Special tiles: They recommended clarifying how "Qu" is handled so it matches the assignment spec.
4. Correctness checks: They confirmed that my minimum word length (3 letters) is handled correctly.

Overall, the feedback was positive. They liked the structure and logic, just small improvements were needed for readability and maintainability.

Feedback I Gave

When I looked at my partner's solver, I highlighted:

1. Good use of sets and prefix pruning for efficiency.
2. Edge cases like malformed grids and multi-letter tiles were mostly handled but a few areas could still break under unusual inputs.

3. Suggested improvements: clearer variable names, less duplicated code and consistent handling of special tiles.

Overall, their code was solid, clear and functional with only minor enhancements needed.

Improvements I made

Based on the feedback, here's what I changed:

1. Helper function for word and prefix sets: I created `_build_sets()` to handle dictionary and prefix setup in one place.
This removed repeated code in `init` and `setDictionary()`.
2. Standardized style: I fixed indentation, cleaned up comments and split long lines for readability.
3. Optimized DFS: I created a `DIRECTIONS` list for all 8 moves so it's easier to read and maintain.
4. Improved clarity: I improved variable names and added comments to explain DFS logic and base cases.
5. Validation: I added more checks for empty or invalid grids to make the code more robust.

Static Analysis Results

Before, linting reported:

1. Module-level imports not at the top (E402)
2. Lines longer than 79 characters (E501)
3. Mixed spaces and tabs in indentation (E101, W191)

After the changes:

1. All imports are at the top, fixing E402.
2. Long lines were split, fixing E501.
3. Indentation is now consistent with 4 spaces so no more mixed spacing issues.

The code now has no linting errors.

Regression Testing

To make sure my changes did not break anything:

1. Ran `getSolution()` on the original 4x4 Boggle board and verified all words were still found.
2. Tested empty grids and boards with “Qu” tiles to check edge cases.
3. Compared outputs before and after refactoring—results were identical.
4. Ensured minimum word length (3 letters) still works.

This confirmed that the refactoring improved style and maintainability without affecting correctness.

Reflection

Doing this code review taught me a lot about how professional workflows work.

1. Readability is important: Even if your code works, clear formatting, comments and modularity make a huge difference.
2. Refactoring pays off: Helper functions and removing duplication make future changes easier.
3. Edge cases matter: Multi-letter tiles, empty boards and other unusual inputs need to be considered.
4. Static analysis helps: Linting caught issues I might not have noticed otherwise.

Overall, I feel more confident reading others' code, giving feedback and improving my own code in a professional way.